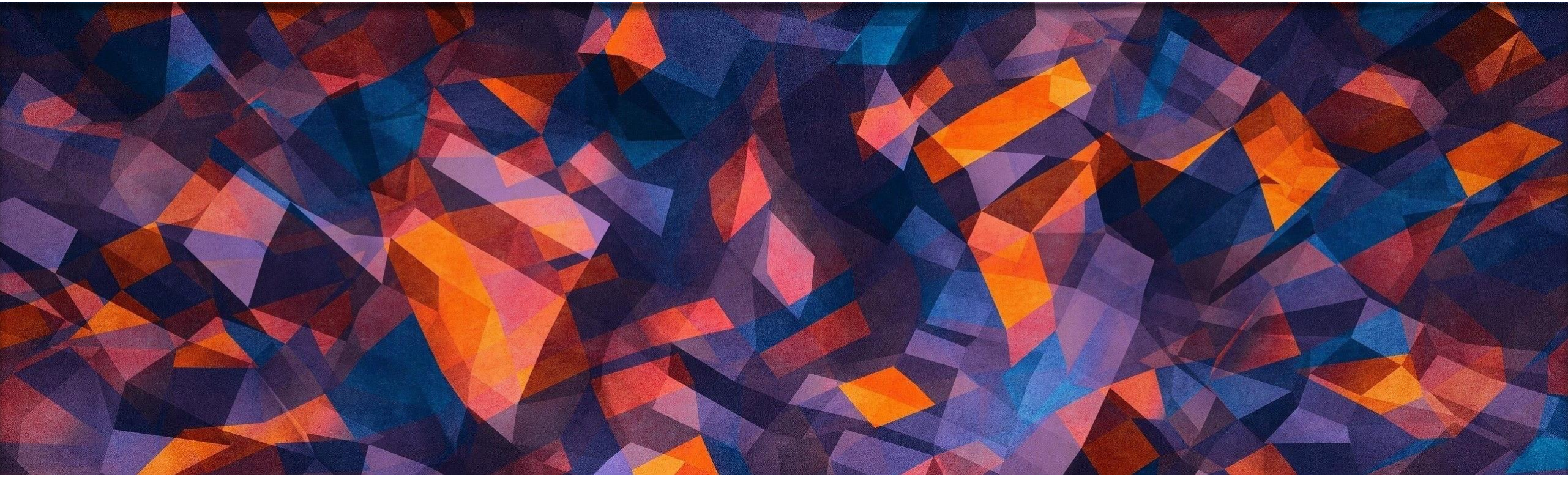
Universität St.Gallen



Event-driven and Process-oriented Architectures

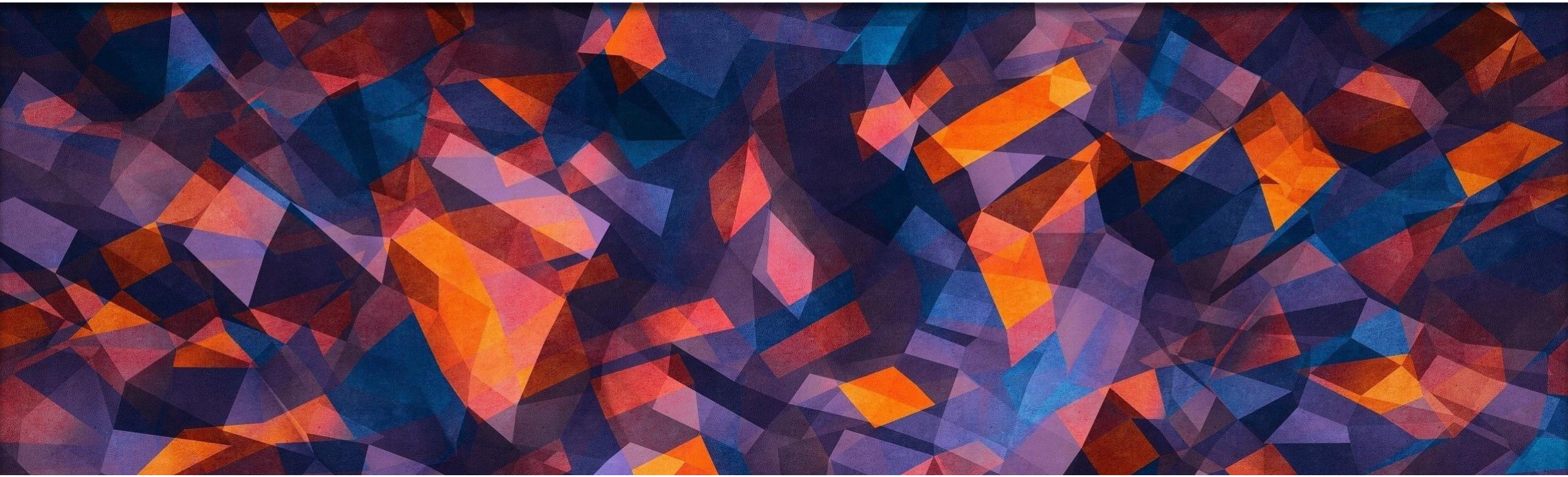
Process Orchestration and Process Automation: Part 2

Lecture 4

Agenda

- Motivating Example: Flowing Retail Example (continued)
- Autonomy, boundaries and isolation: Avoiding process monoliths
- Balancing orchestration and choreography
- Workflow engines and integration challenges

Universität St.Gallen

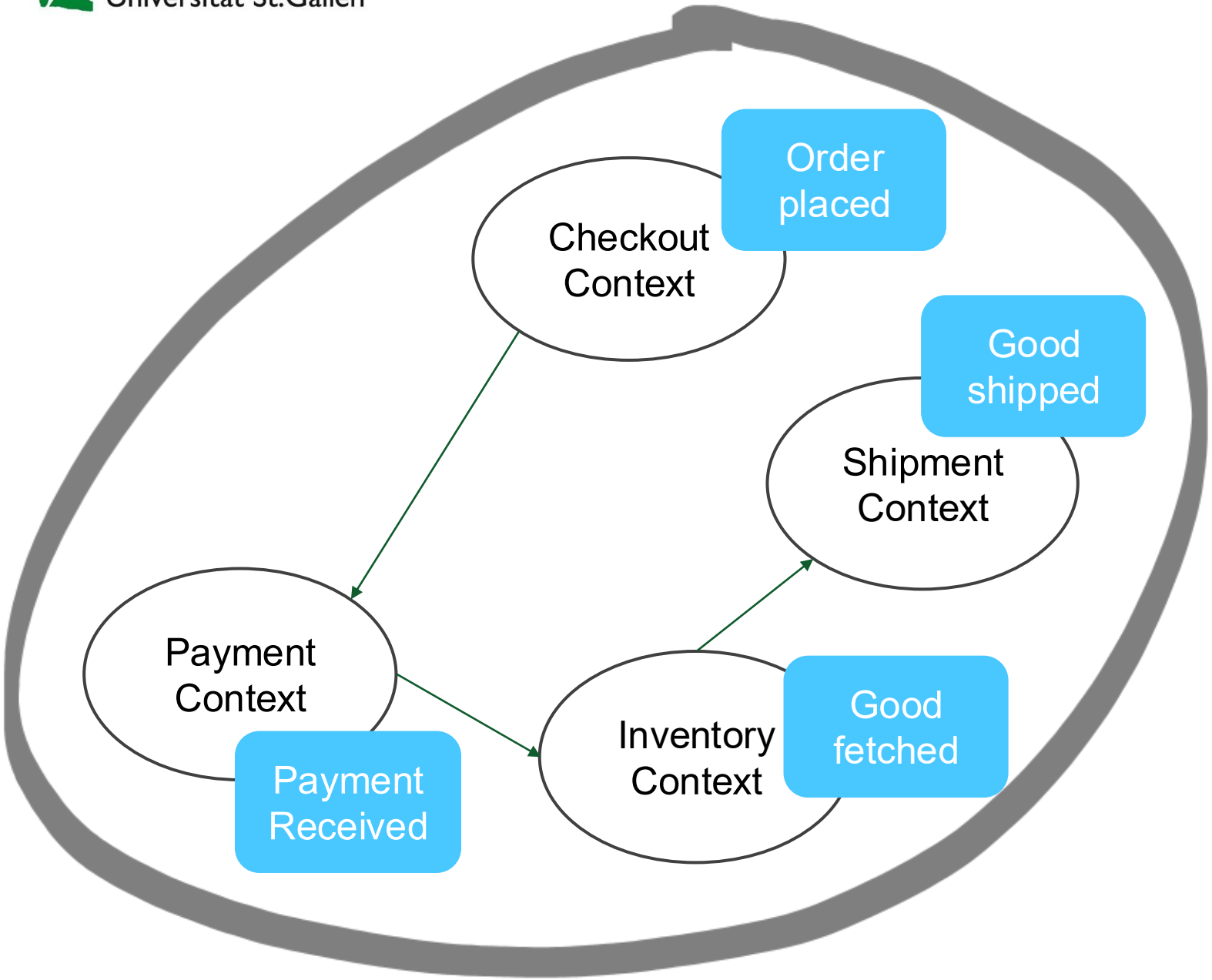


Event-driven and Process-oriented Architectures

Motivating Example

Lecture 4

Order Fulfillment Flow
Choreography

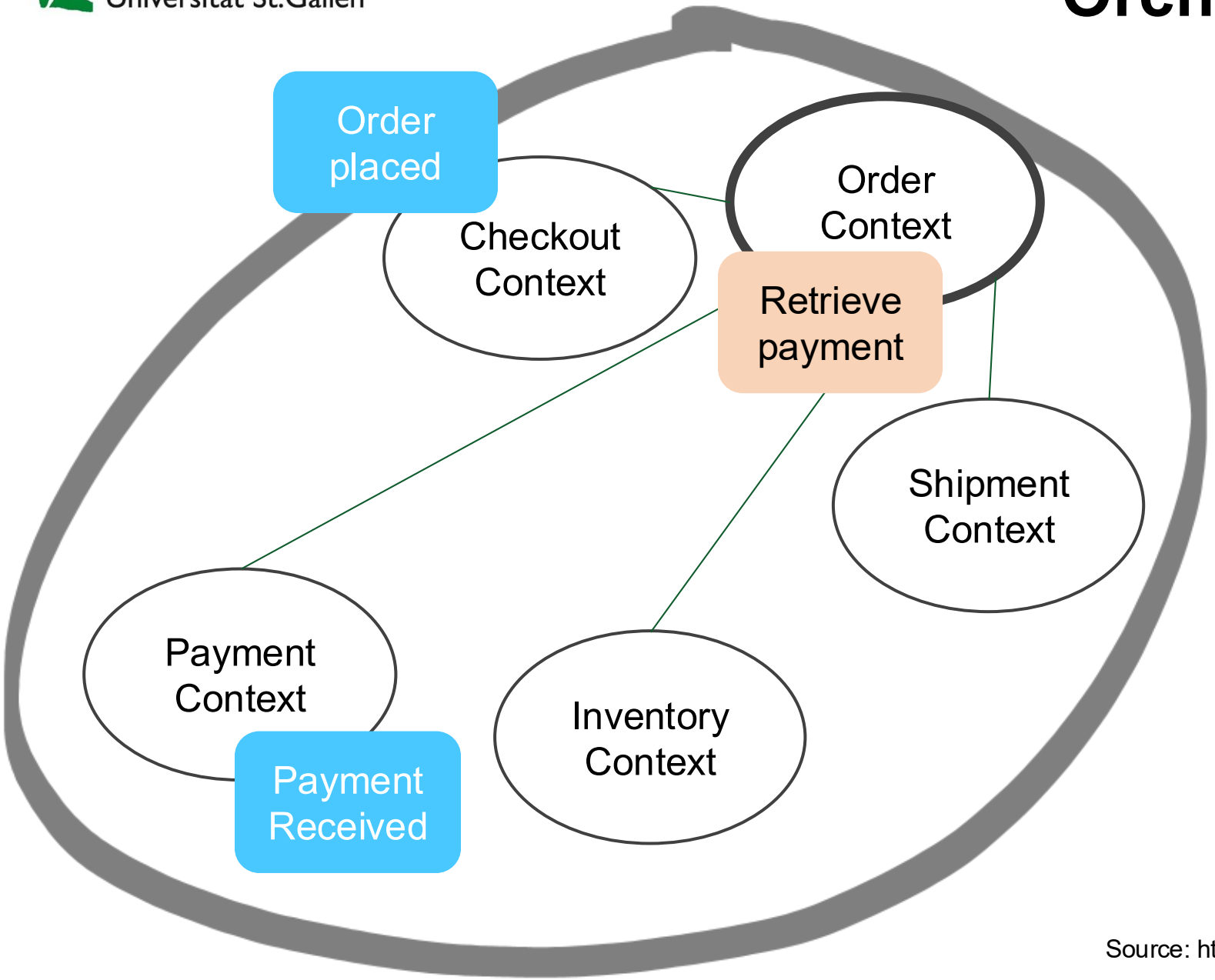


In choreography the **process emerges from event chains.**

No single component owns or controls the end-to-end flow.

(Week 2)

Orchestrating the Flow



Orchestration introduces an explicit process owner.

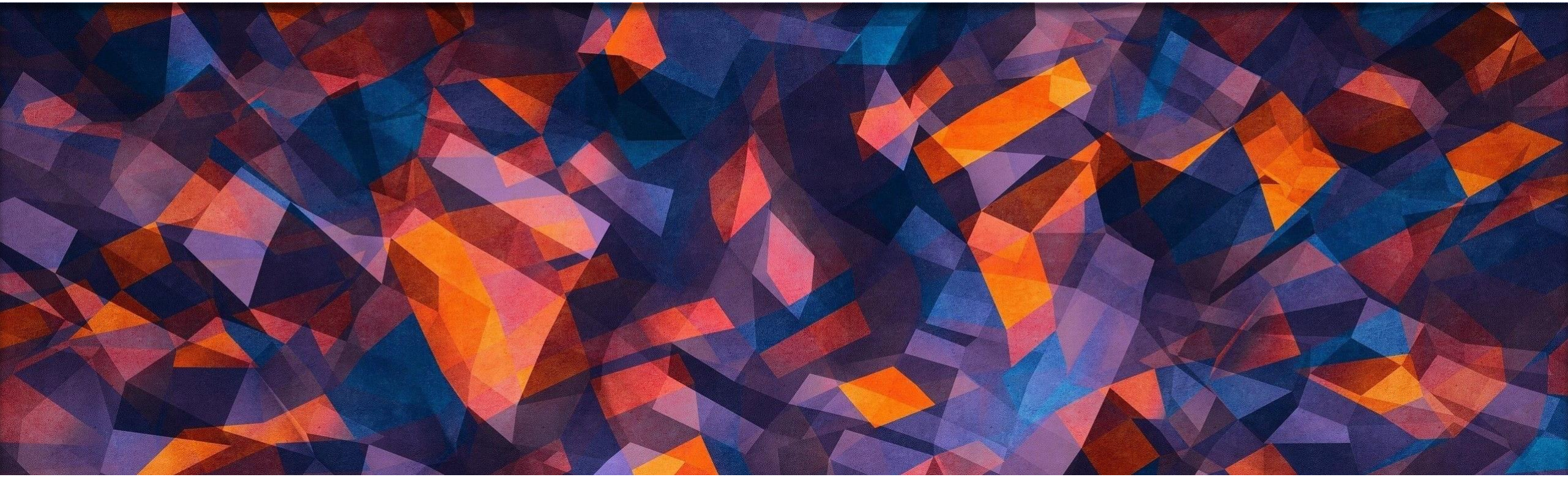
The workflow becomes a first-class artifact.

(Week 3)

In microservice architectures this raises new questions:
- Where should the workflow live?
- How do we respect service boundaries?

Source: https://www.infoq.com/articles/events-workflow-automation/

Universität St.Gallen



Event-driven and Process-oriented Architectures

Autonomy, Boundaries and Isolation: Avoiding the Process Monolith

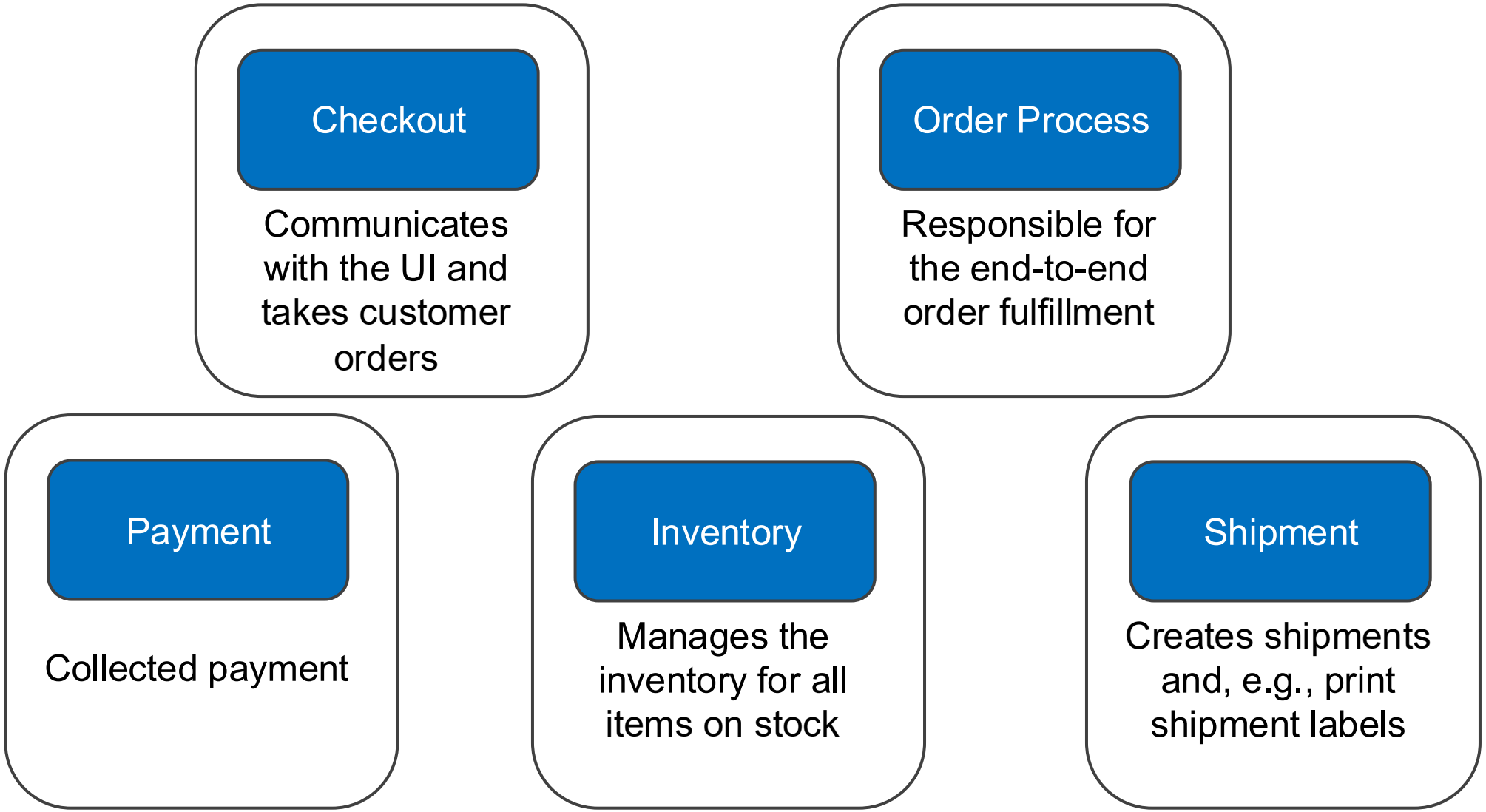
Lecture 4

Autonomy and Isolation
Valued in Microservices Architectures

Key Question.
How should process ownership align with service boundaries

- Services should align with bounded contexts (see ASSE)
- Proceses often cross multiple contexts
- Workflow engines must respect service autonomy

Domain Driven Design, Bounded Contexts, Services



Boundaries and Business Processes

Business processes often span multiple bounded contexts.

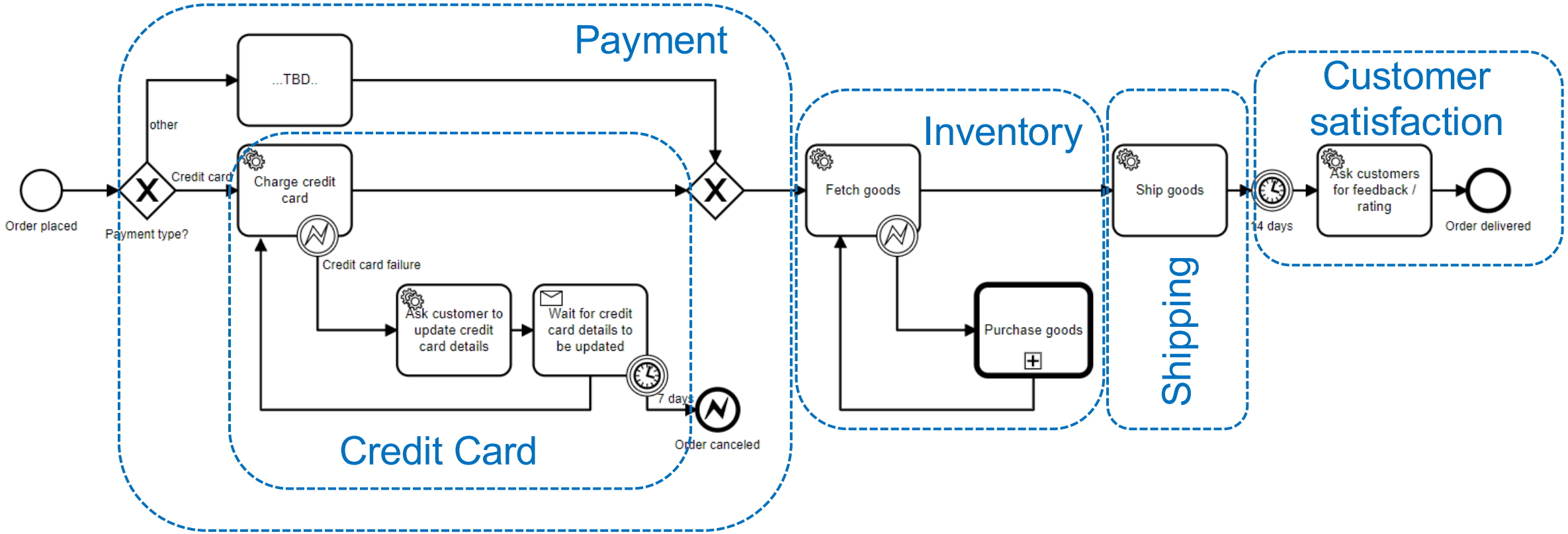
- The process model should have a single owner
- That owner shoudl be one bounded context

Boundaries and Business Processes

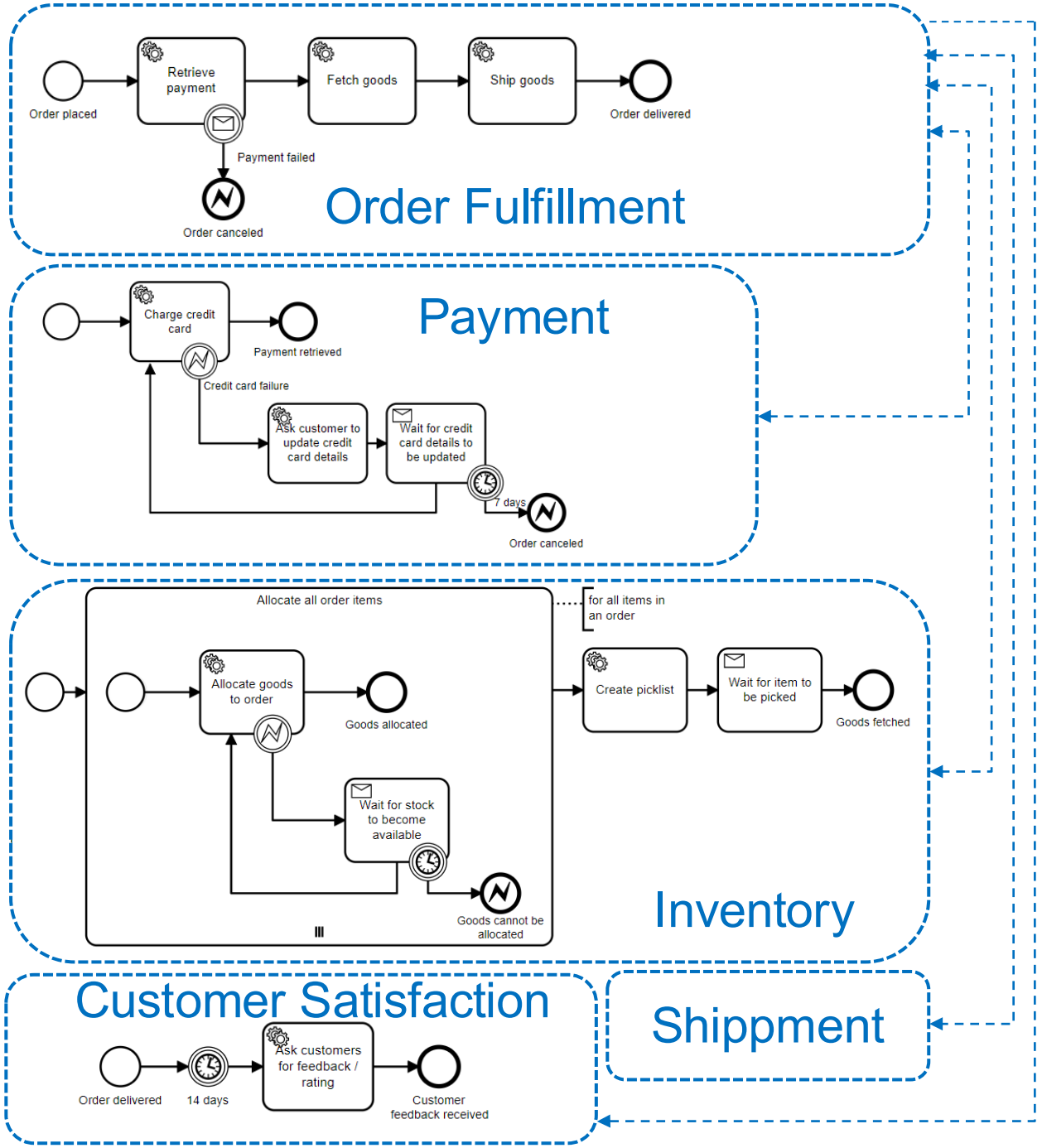
- Modeling business processes can help to **identify boundaries** (outside of the scope of this lecture)
- Workflow engines help manage **long-running processes** without breaking service boundaries

Respect Boundaries - Avoid Process Monolith

One process model spanning multiple services creates a process monolith



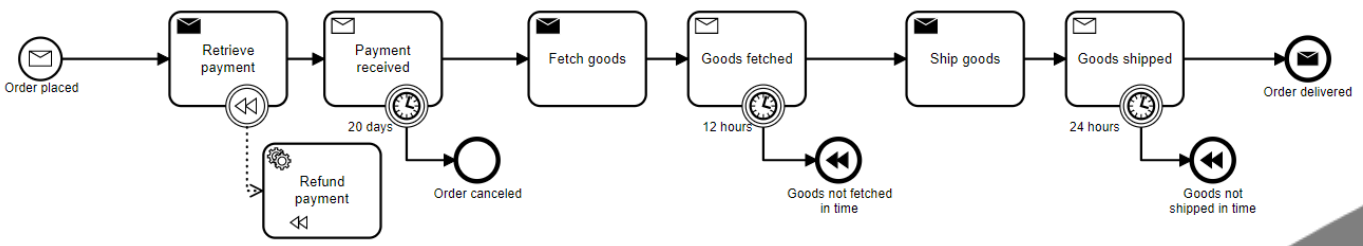
Respect Boundaries - Avoid Process Monolith

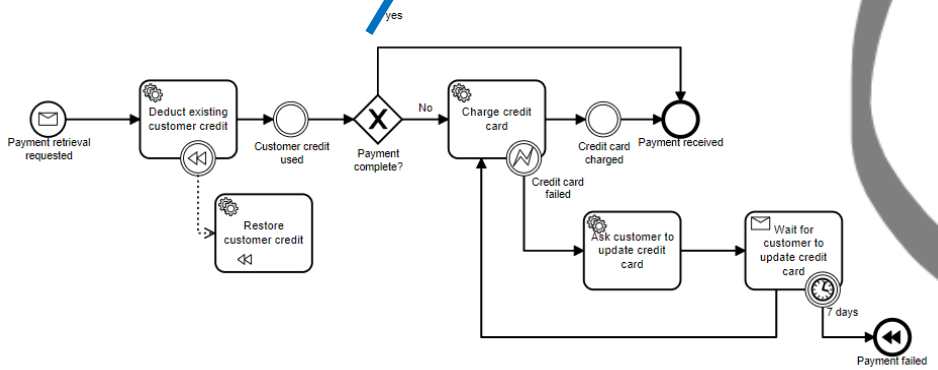


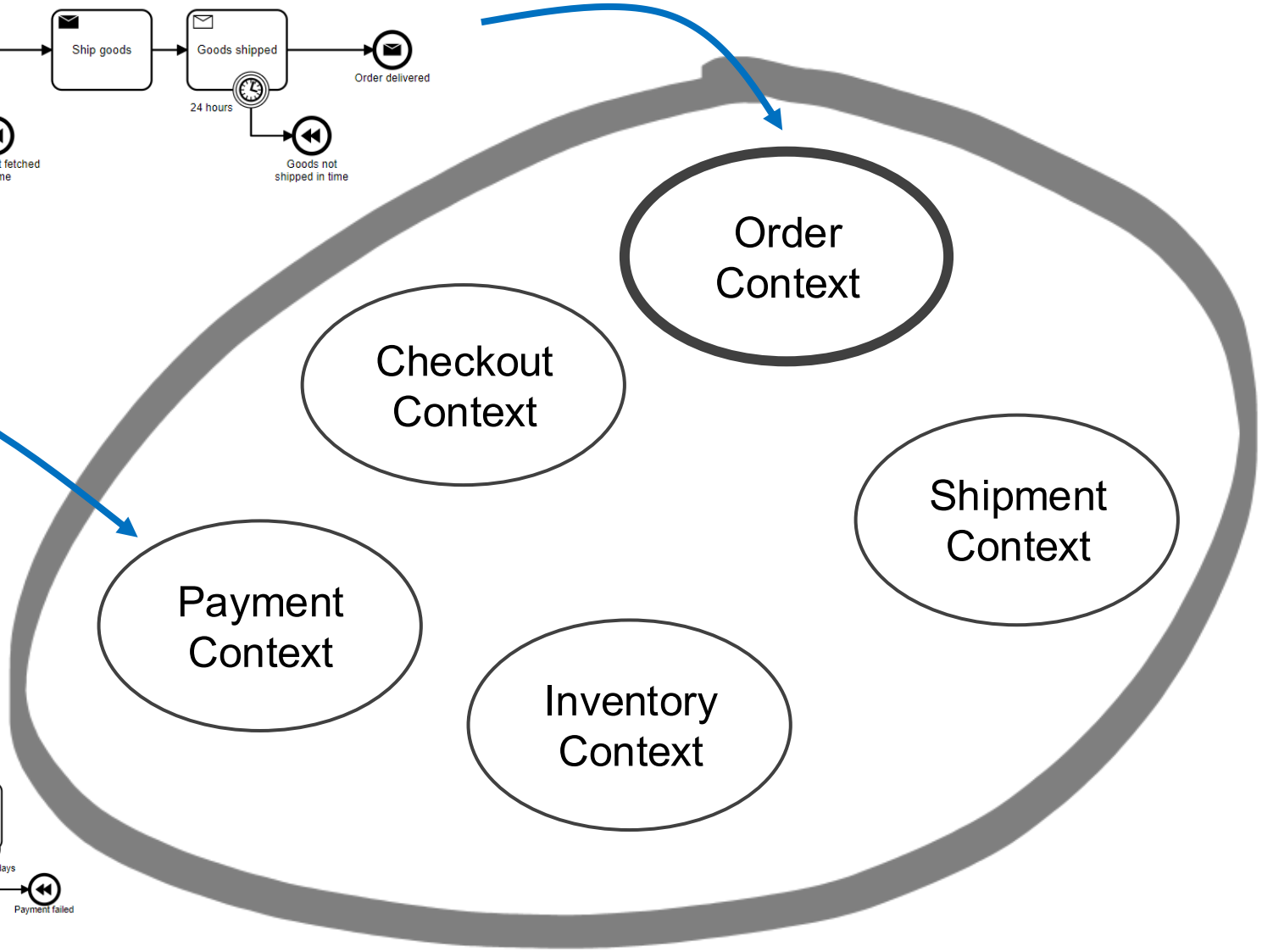
- Each bounded context owns its part of the workflow process logic
- Distribute responsibilities to maintain service autonomy and team scalability
- Leads to the question: central versus decentralized workflow engine

From Monoliths to Microservies

End-to-end processes are split into service-owned workflow fragments







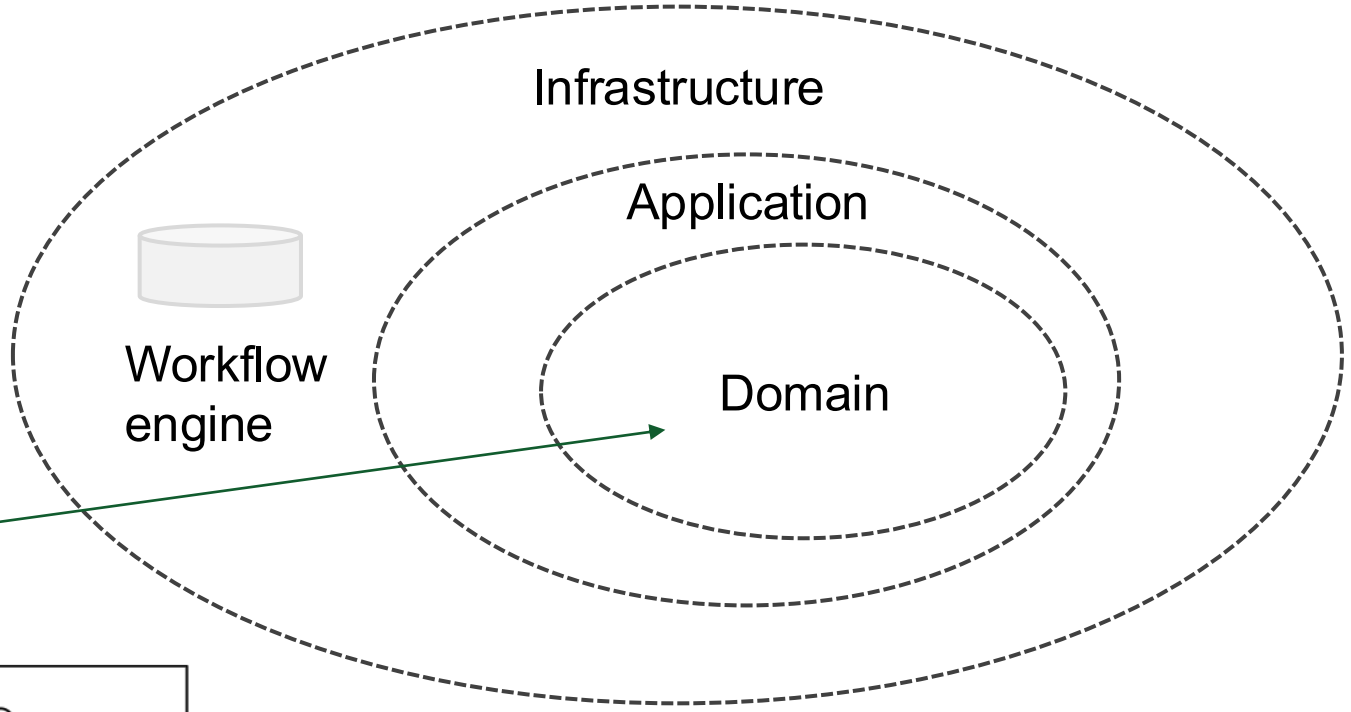
From Monoliths to Microservies

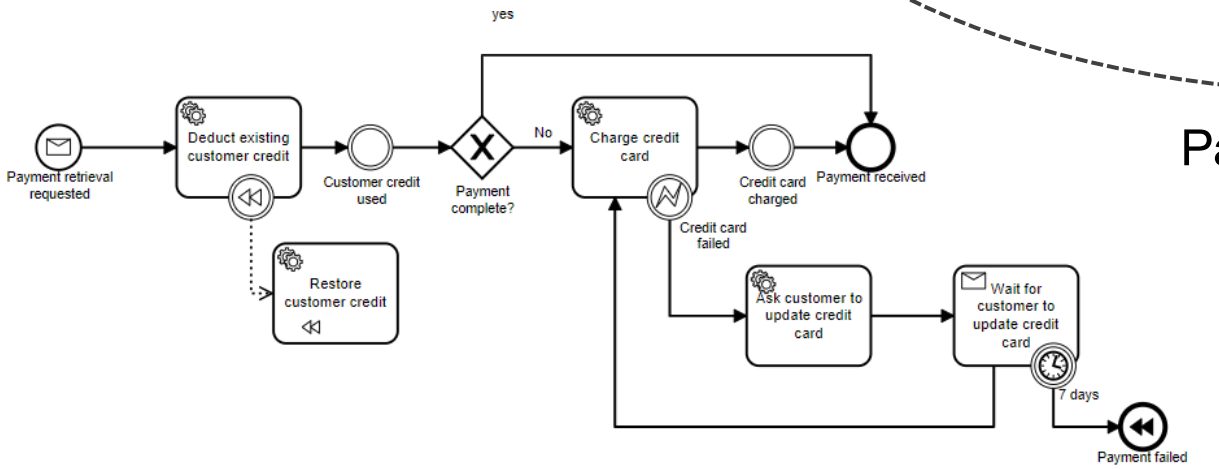
Workflow Logic Lives Inside the Service

Domain Logic

+

Flow



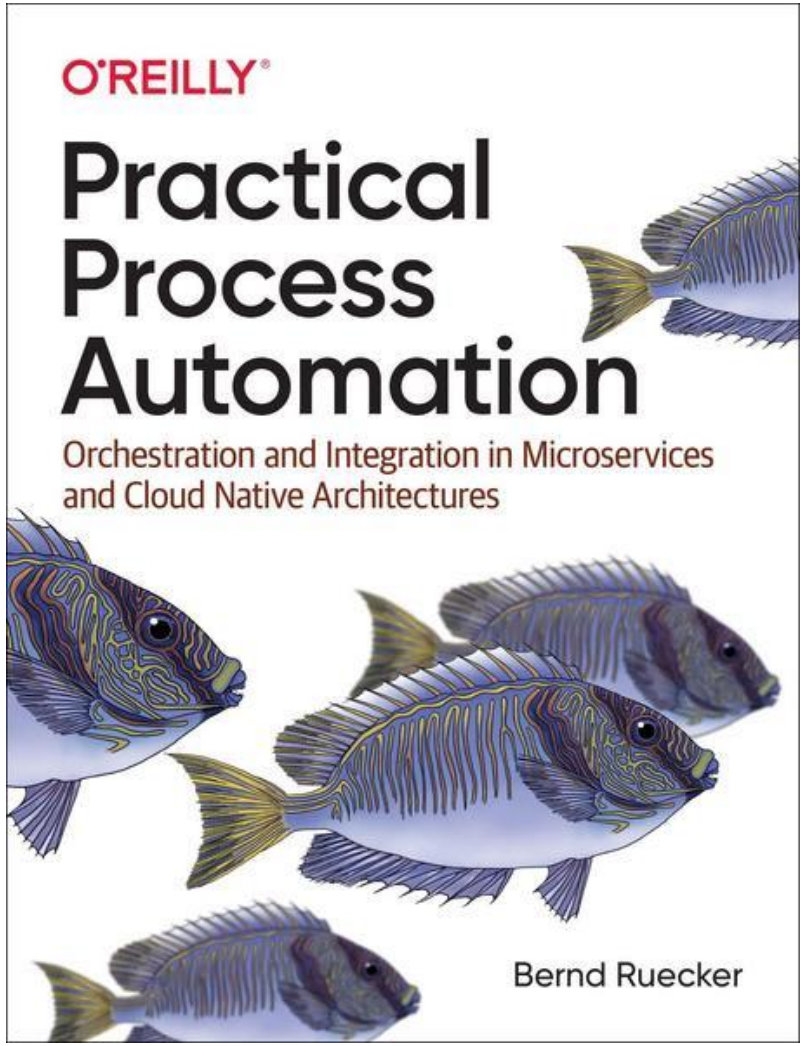


Payment Service

The workflow engine executes the flow that orchestrates domain operations.

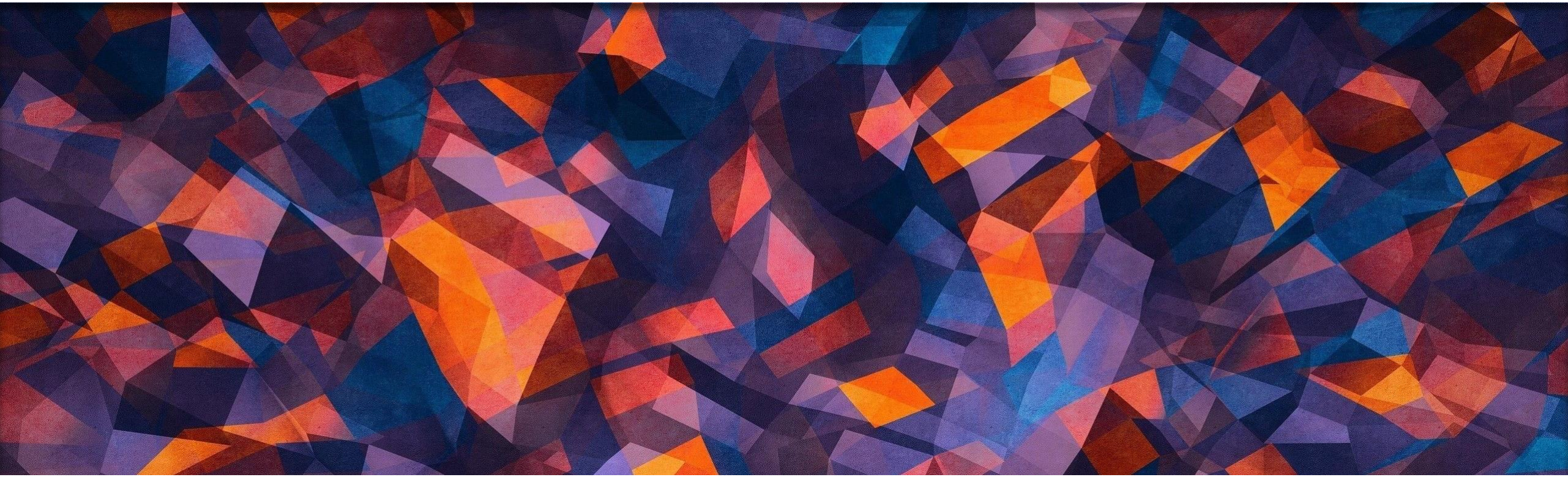
Source: https://www.infoq.com/articles/events-workflow-automation/

Further References



Based on Chapter 7 – Autonomy, boundaries and isolation

Universität St.Gallen



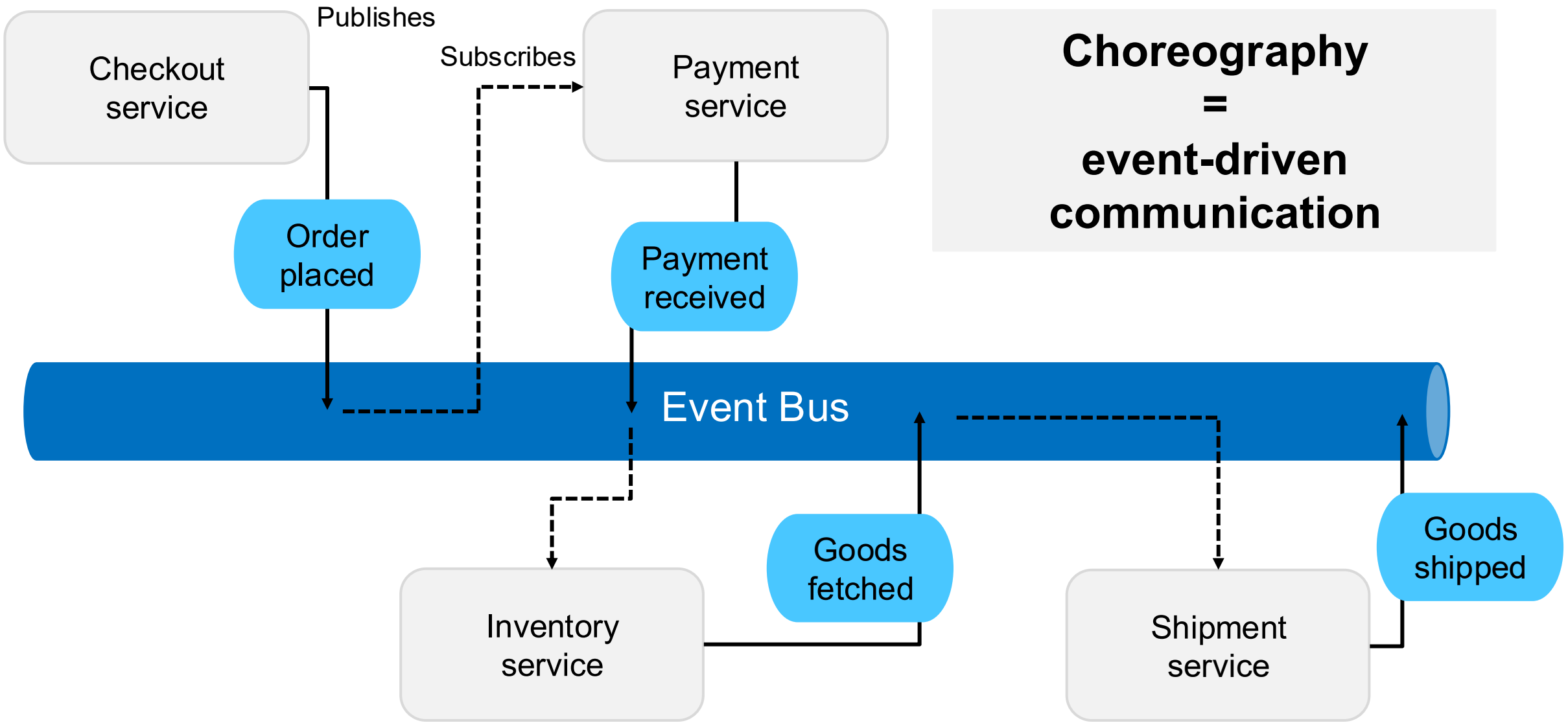
Event-driven and Process-oriented Architectures

Balancing Orchestration and Choreography:

Designing Commands and Events

Lecture 4

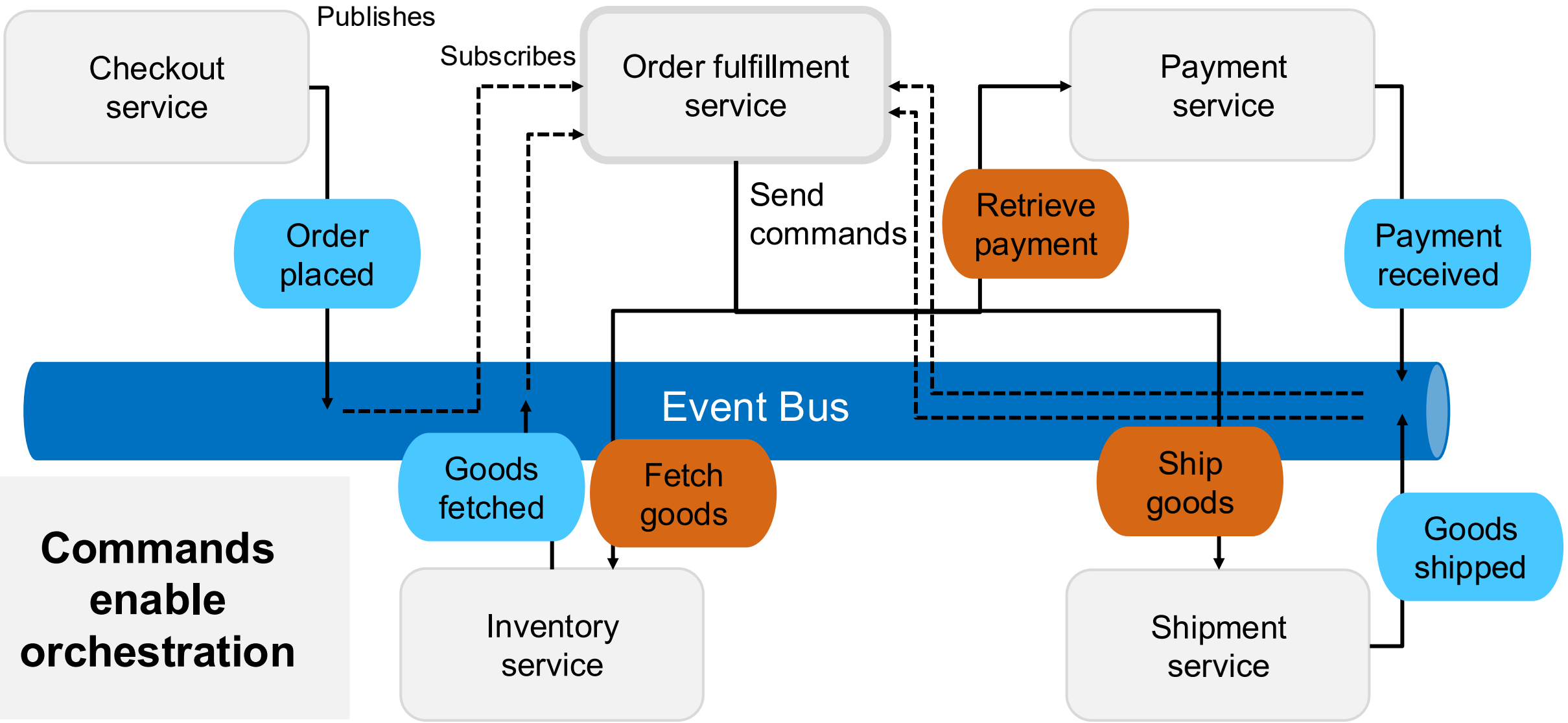
Recap: Choreography is Event-Driven



Choreography = event-driven communication

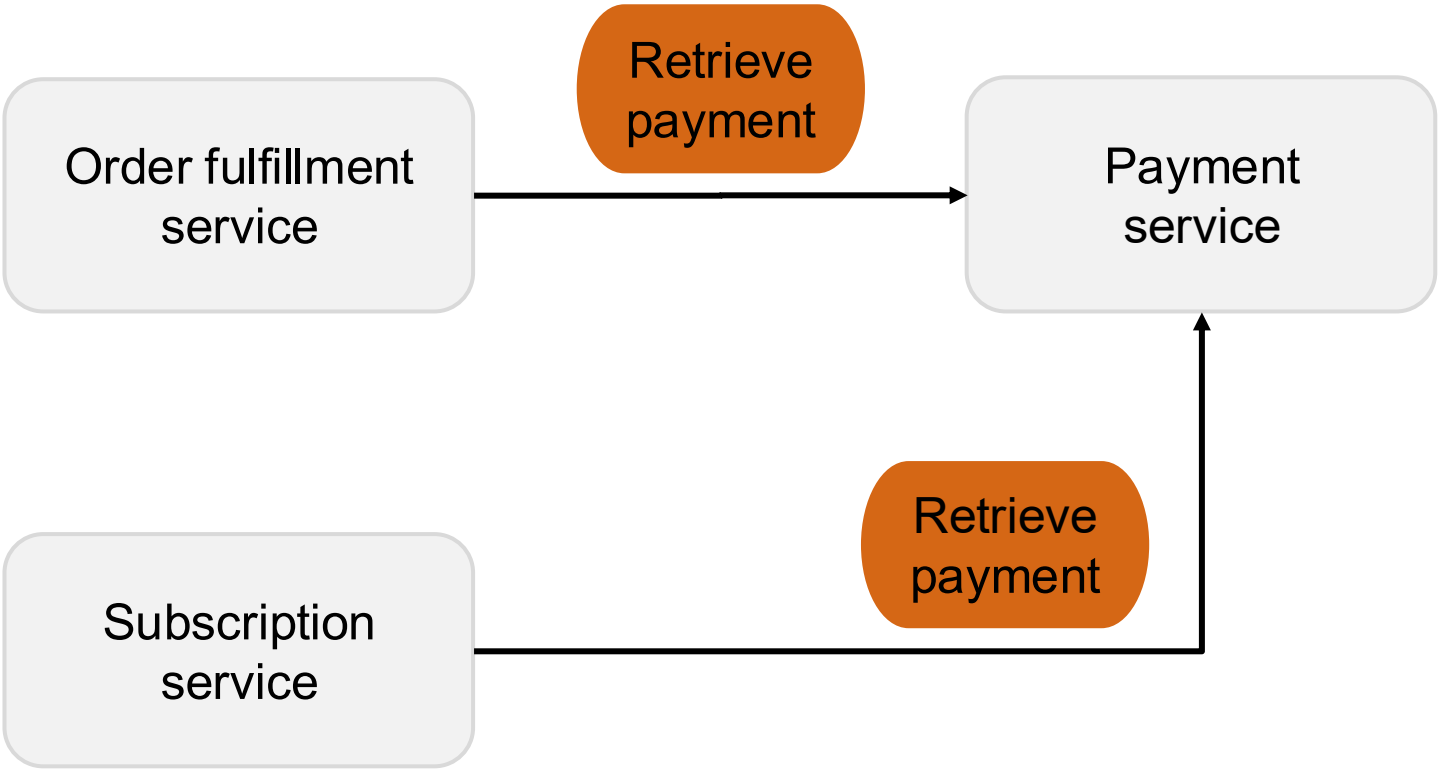
Breaking Event Chains With Commands

Commands make responsibilities explicit in long event chains



Commands enable orchestration

Commands Make Responsibilities Explicit



Payment owns payment responsibility.

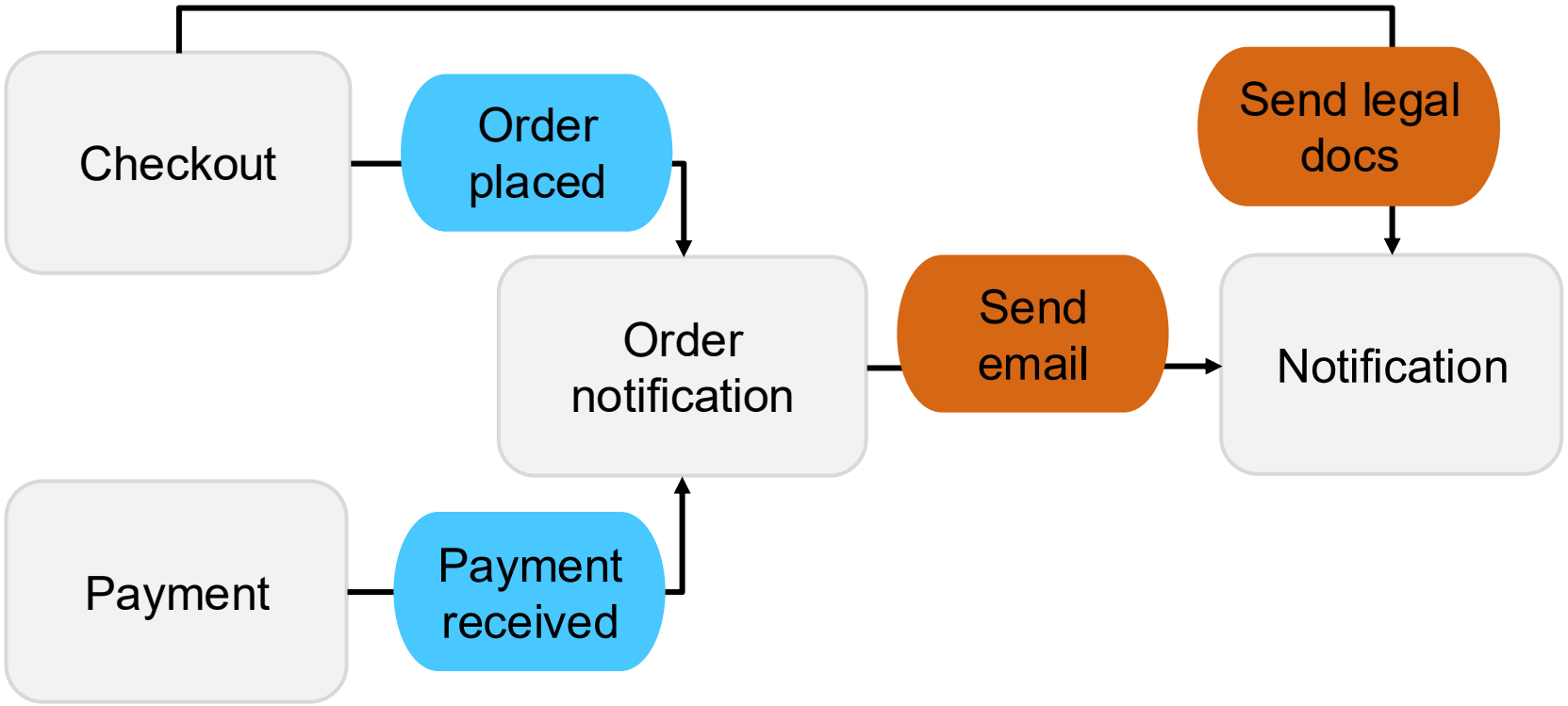
Payment only reacts to **payment commands** (e.g., *Retrieve payment*).

It does not **need to understand foreign events** like *Order placed*.

If payment reacted to events like “Order placed”, adding new services could require changes to Payment.

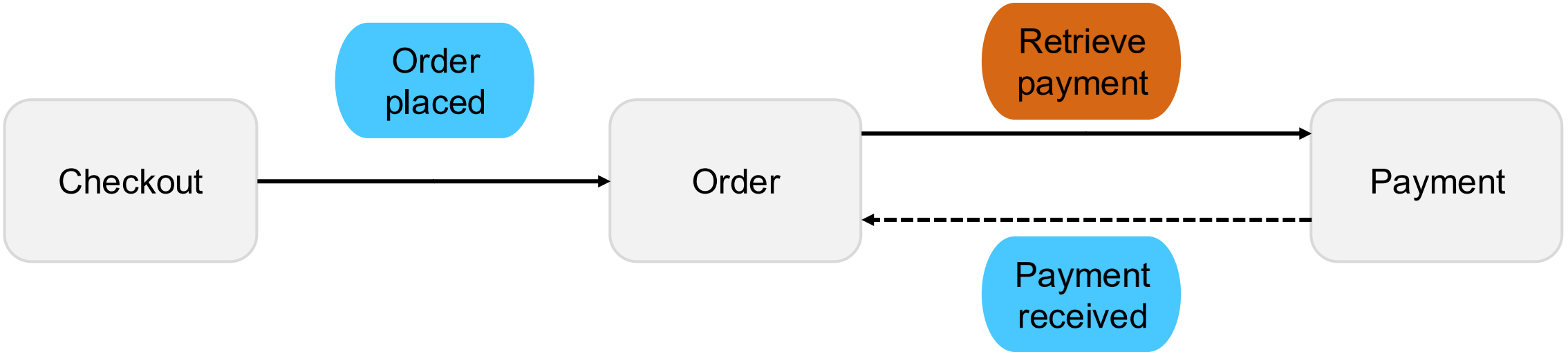
Note: Generic services should not change when new consumers appear.

Notifications: Events or Commands



Responsibility determines whether to use commands or events.

The Direction of Dependency



Event-driven
Receiver decides to couple

Command-driven
Sender decides to couple

Direction of Dependency

Finding the Right Balance

How to choose between commands and events

- Who is responsible for the action?
- Where should coupling be controlled?
- Can the event safely be ignored?
- What happens if new consumers appear?

A Simple Rule for Events vs Commands

- Is it OK with the component emitting an event if that event is ignored?

Sending notifications in the order fulfillment example

Receiving side is **responsible** for taking action

Yes

It really is an event

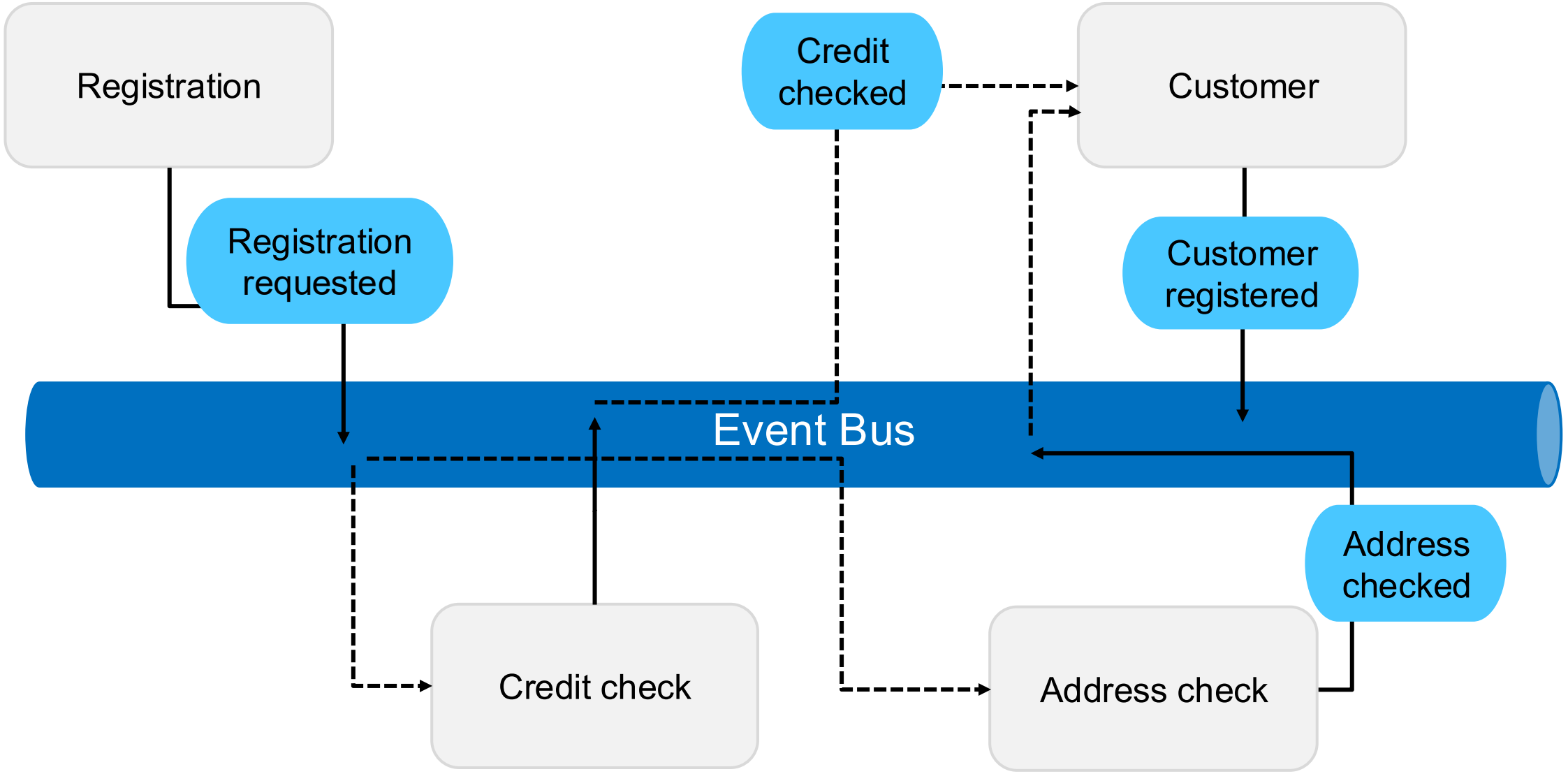
No

You probably have a command in front of you

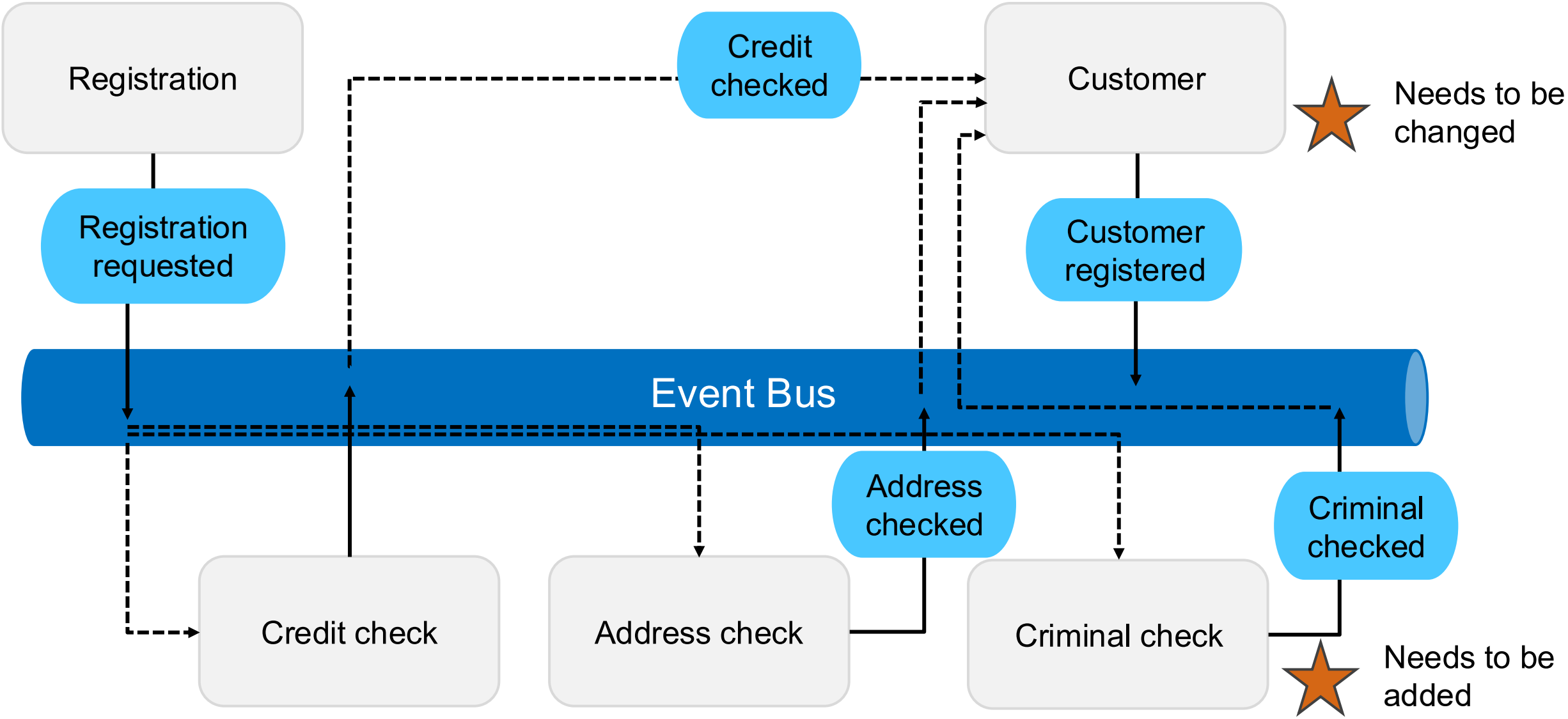
Sending notifications required by law

Sending side is **responsible** that something is happening

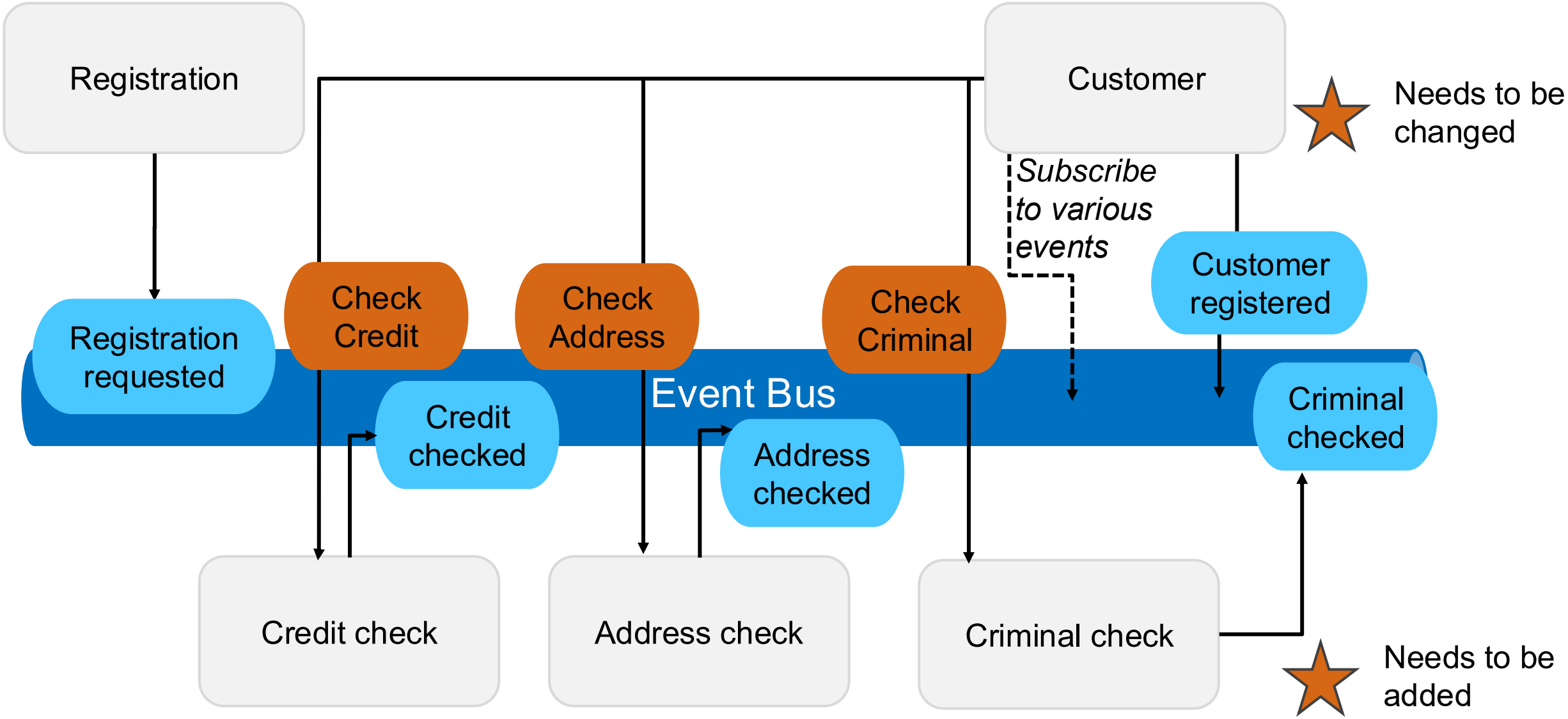
Evaluating Change Scenarios
Customer Onboarding Process



Evaluating Change Scenarios
Customer Onboarding Process - Choreography

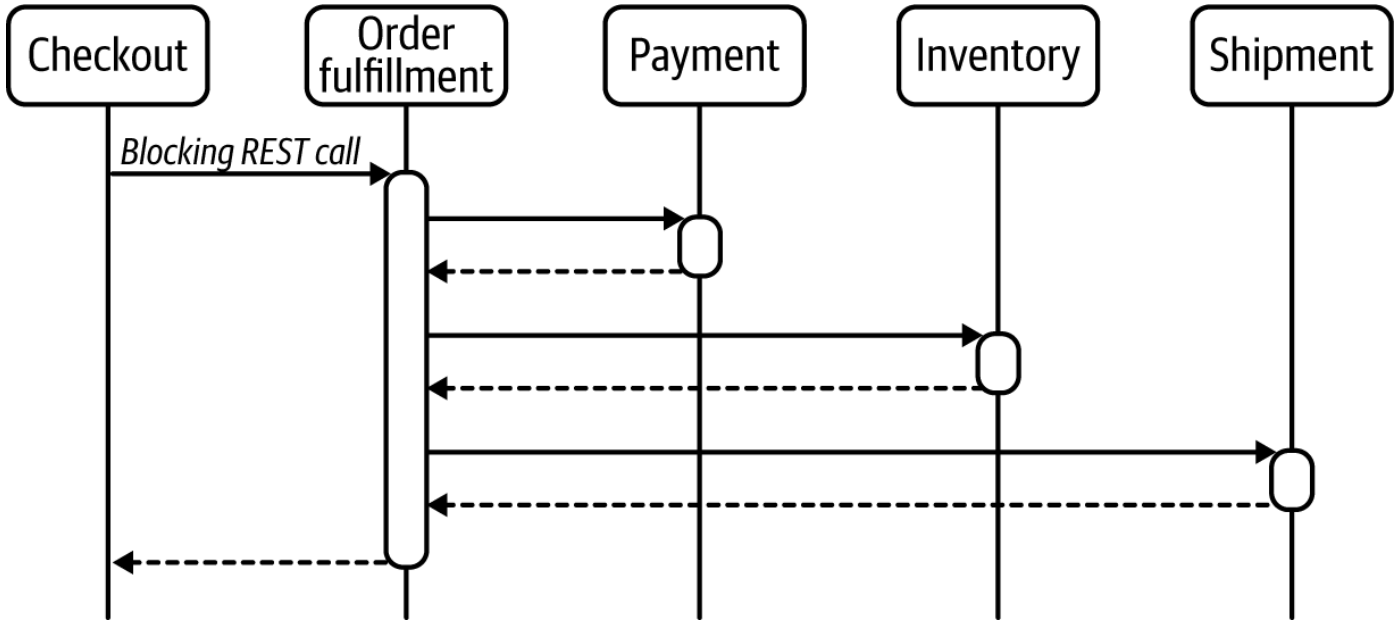


Evaluating Change Scenarios
Customer Onboarding Process - Orchestration



Common Misconception
Commands Require Synchronous Communication

Temporal coupling comes from synchronous communication, not commands



Synchronous communication creates temporal coupling

Common Misconception
Orchestration Needs to be Central

- Orchestration means sending *commands* to another component

- Historically (SOA / ESB):
  - Orchestration was often centralized

- In microservices:
  - Orchestration can be local to a service
  - Or distributed across services

Key takeaway: **Orchestration is a pattern, not a deployment model.**

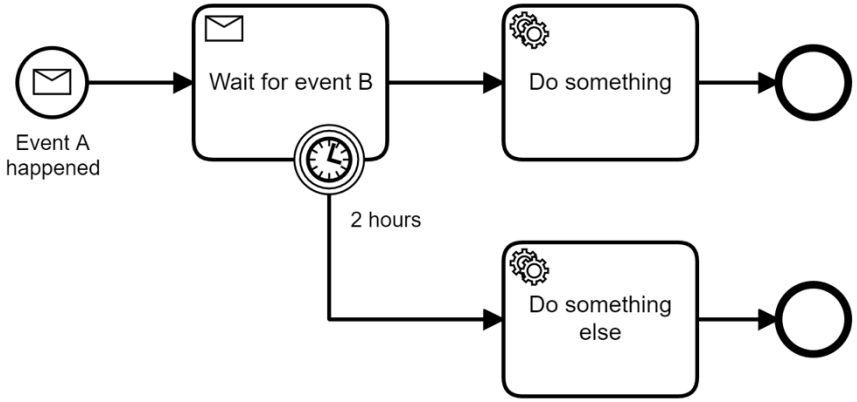
Common Misconception
Choreography Leads to More Decoupling

- Any communication between services introduces some form of coupling

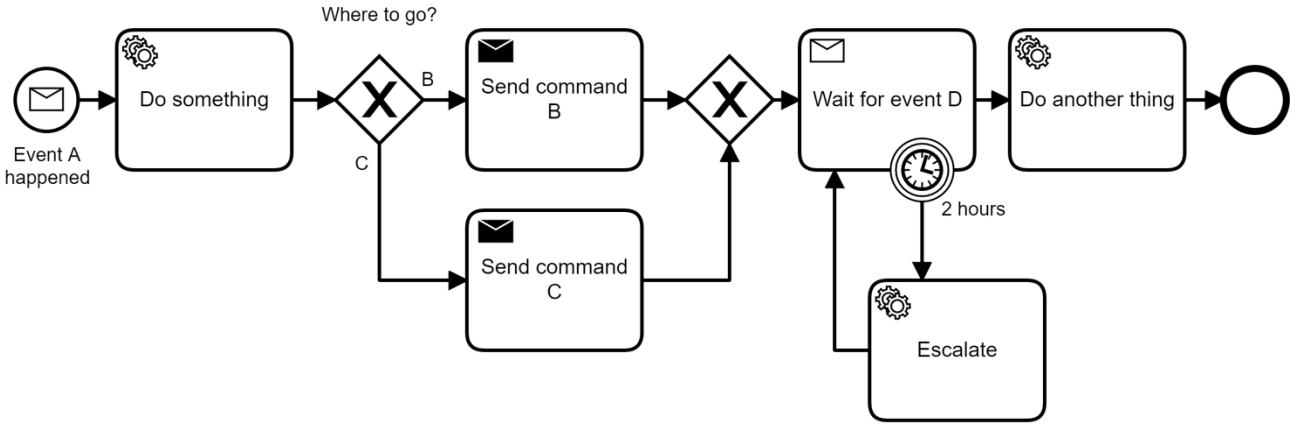
- Events move the coupling decision to the receiving side.
  - This can increase flexibility — but also makes dependencies less explicit.

Workflow Engines Combine Events and Commands

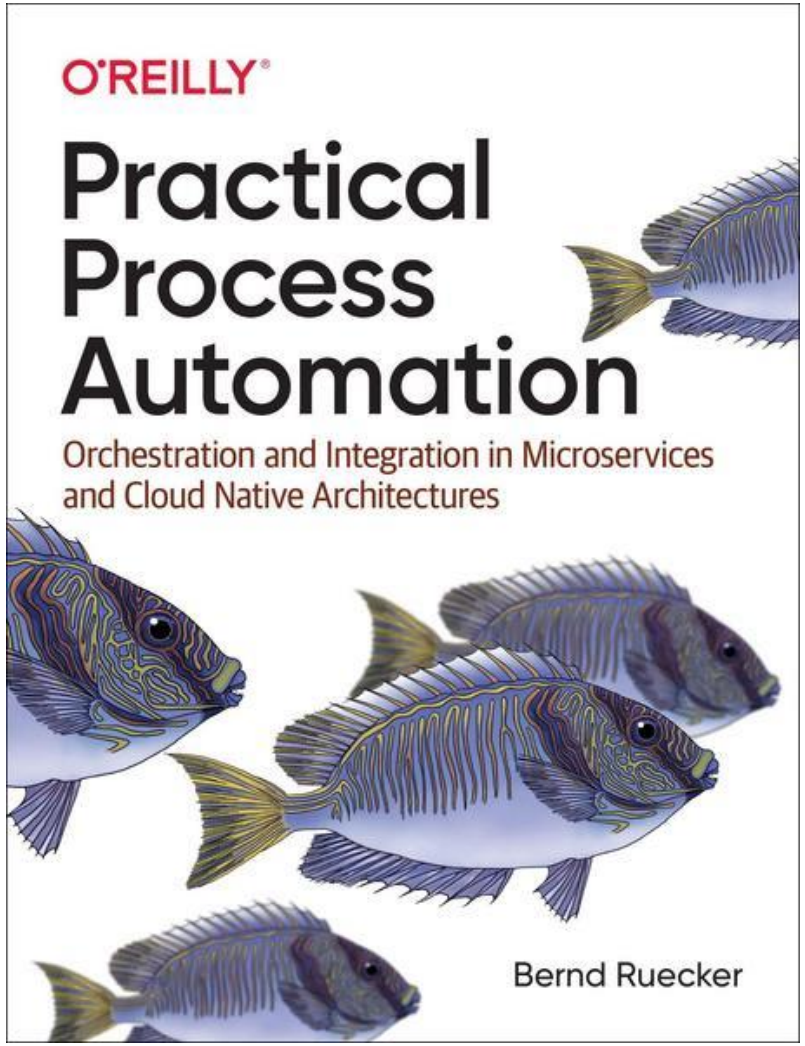
Event-driven workflow



Events + Commands



Further References



Based on Chapter 8 – Balancing Orchestration and Choreography

Camunda 7 Vs. Camunda 8: Key Differences

Aspect	Camunda 7	Camunda 8
Engine Architecture	Allows embedding the workflow engine as a **library** in your application	The workflow engine in Camunda 8, Zeebe, is always a **remote resource** for your application
Saga Pattern Support (will be covered next week)	Supported through BPMN compensation events	**Also supported** through BPMN **compensation events** since Camunda 8.5
Persistente Model	Engine state stored directly in **DB tables**	Zeebe leverages **event sourcing**

Camunda 7 Vs. Camunda 8: Key Differences

Aspect	Camunda 7	Camunda 8
Glue Code to Service Tasks	Supports **Java delegates** and **workers**	Support **workers**
Connectors	Integration relies mainly on **custom code or REST APIs** that **must be configured and implemented manually.**	Provides **ready-to-use connectors** for common services and APIs (e.g., **ChatGPT, Email, Slack**) https://docs.camunda.io/docs/components/connectors/introduction-to-connectors/

Camunda 7 Vs. Camunda 8: Key Differences

Aspect	Camunda 7	Camunda 8
Deployment Simplicity	Since it can run as a **single application with an embedded engine**, this makes it **easy to deploy and operate in smaller systems**	Requires a **distributed architecture** with Zeebe, Operate, Tasklist, and other components
Community Support	Fully open-source ecosystem with long-standing and extensive community support, **now led by Operaton**	Newer platform, **not fully open-source**, oriented toward SaaS, with a rapidly evolving ecosystem.

Given all these arguments, **consider which option best fits the context and requirements of your project.**

References

- https://docs.camunda.io/docs/guides/
- https://docs.camunda.io/docs/guides/migrating-from-camunda-7/conceptual-differences/
- https://docs.camunda.io/docs/components/connectors/introduction-to-connectors/
- https://camunda.com/blog/2022/05/how-open-is-camunda-platform-8/

Universität St.Gallen

Event-driven and Process-oriented Architectures

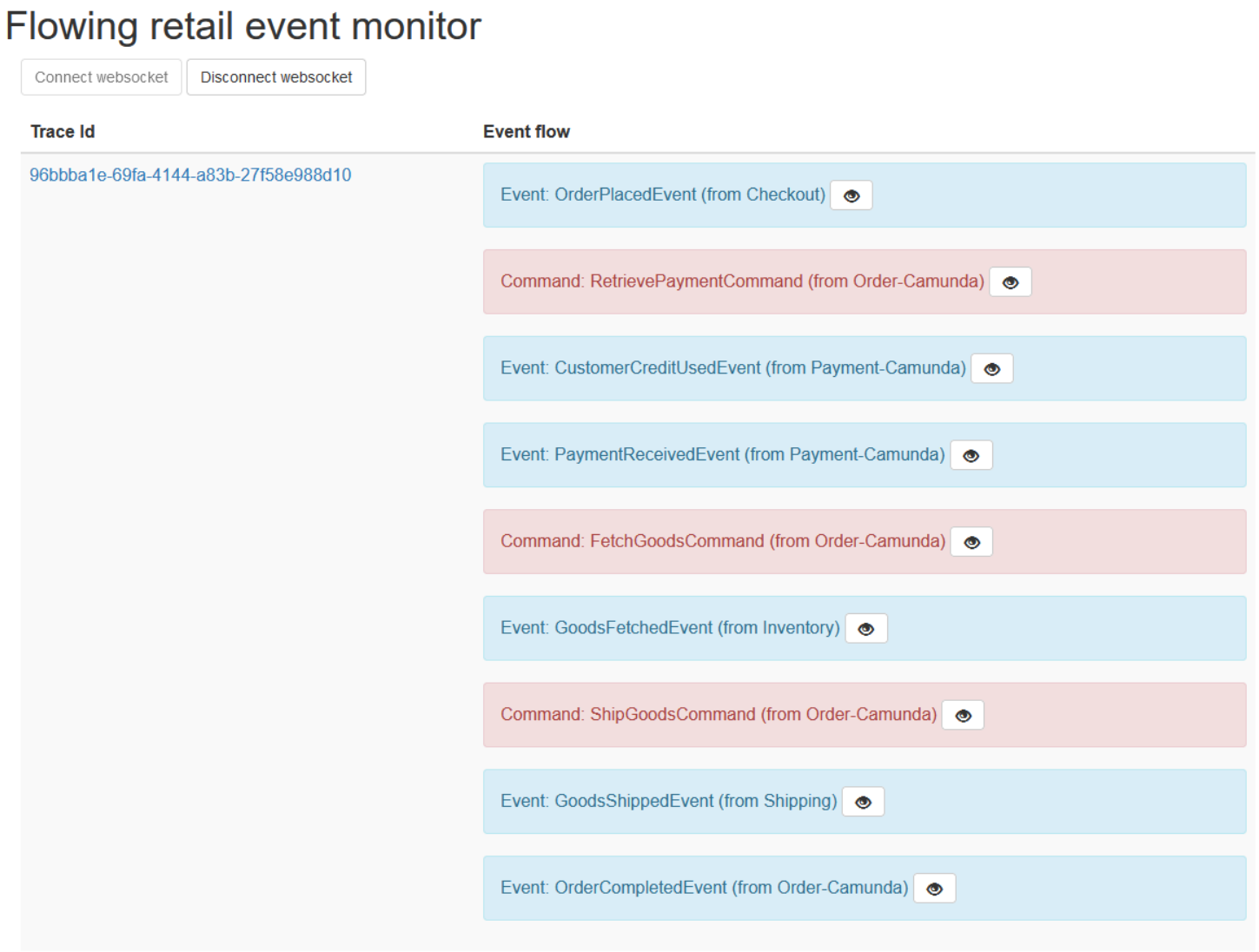
Exercise

Lecture 4

- Lab 7: Flowing Retail Camunda Version
  - Lab 4 addresses the Choreography version of the *flowing retail* implementation
  - Study the *Kafka-Java-Order-Camunda* version:
    - https://github.com/scs-edpo/lab04-flowing-retail/tree/master/kafka/java with
      - https://github.com/scs-edpo/lab04-flowing-retail/tree/master/kafka/java/order-camunda
      - https://github.com/scs-edpo/lab04-flowing-retail/tree/master/kafka/java/payment-camunda
  - Find out what has changed compared to the pure choreography-based version
  - Discuss what the trade-offs and consequences are
  - Try to trigger some of the exceptions/timeouts in the BPMN models

Universität St.Gallen

- Lab 7: Flowing Retail Camunda Version



Labs: Zeebe.io

Lab 10: Camunda - Cloud Get Started

https://github.com/scs-edpo/lab10-camunda-cloud-get-started

Blog Posts:

- https://blog.bernd-ruecker.com/zeebe-io-a-horizontally-scalable-distributed-workflow-engine-45788a90d549
- https://blog.bernd-ruecker.com/how-we-built-a-highly-scalable-distributed-state-machine-f2595e3c0422

Labs: Zeebe.io

Lab 11a: Flowing Retail – Kafka Zeebe Version

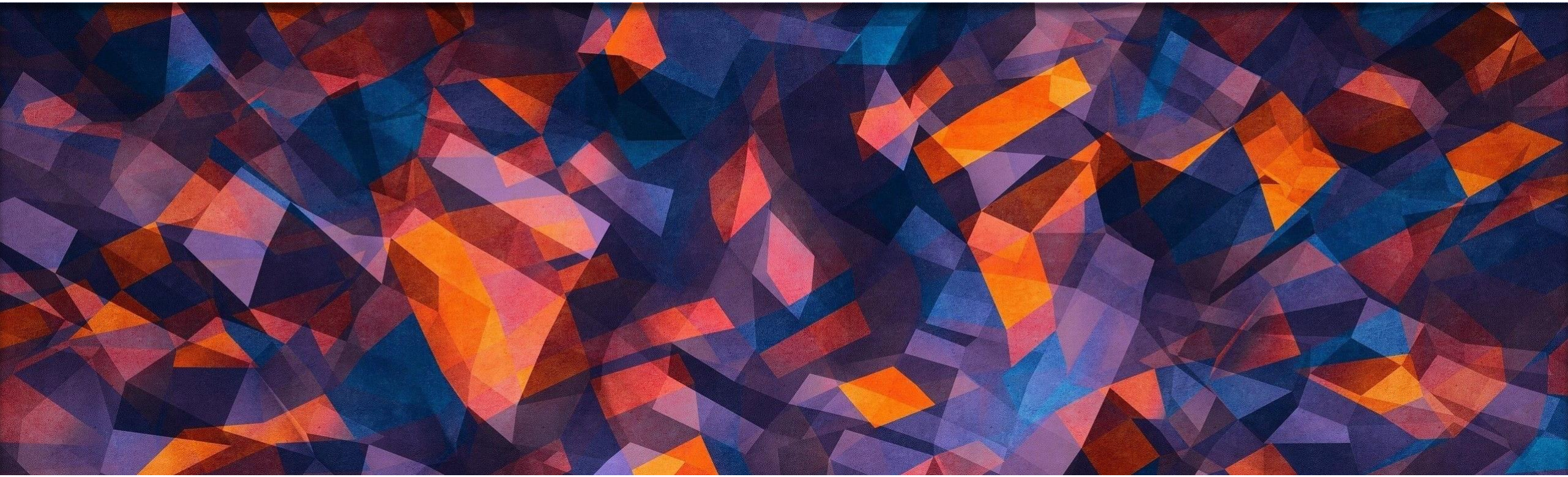
https://github.com/scs-edpo/lab04-flowing-retail/tree/master/kafka/java

https://github.com/scs-edpo/lab04-flowing-retail/tree/master/kafka/java/order-zeebe

Lab 11b: Flowing Retail – Zeebe Version without Kafka

https://github.com/scs-edpo/lab04-flowing-retail/tree/master/zeebe

Universität St.Gallen

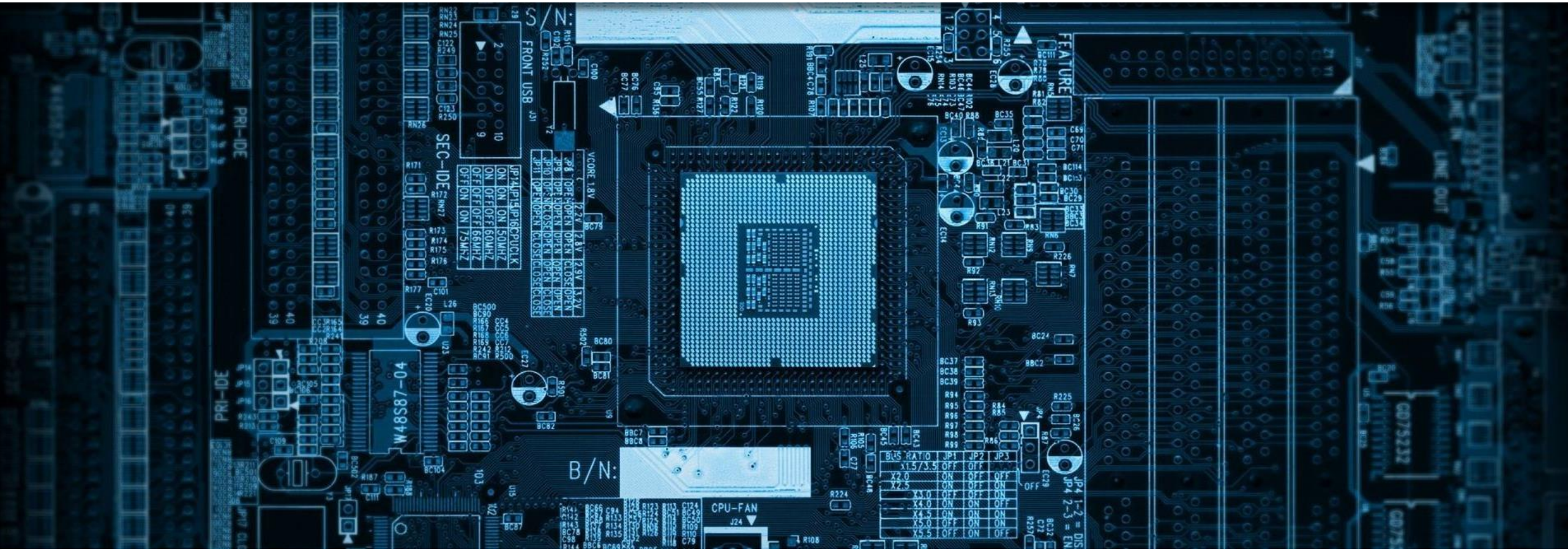


Event-driven and Process-oriented Architectures

Ideas for Project

Lecture 4

- Extend the order flow with additional events and commands
- Develop a prototype of an event-driven system where you try to find a balance between events and commands
- Decide about using Camunda 7 or Camunda 8



Questions?